



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**DEVELOPMENT OF AN ALGORITHMIC FINANCIAL PORTFOLIO MANAGEMENT SYSTEM
FOR SQL DATA ACQUISITION, NORMALIZATION, AND PERFORMANCE PROFILING**

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA1213 - PYTHON PROGRAMMING

to the award of the degree of

**BACHELOR OF ENGINEERING IN
COMPUTER SCIENCE ENGINEERING**

Submitted by

DILEEP KUMAR DESHABOINA (192424281)

Under the Supervision of Dr.Loganayagi

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical
Sciences Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “DEVELOPMENT OF AN ALGORITHMIC FINANCIAL PORTFOLIO MANAGEMENT SYSTEM FOR SQL DATA ACQUISITION, NORMALIZATION, AND PERFORMANCE PROFILING” has been carried out by DILEEP KUMAR DESHABOINA (192424281) as part of the three-module team project under the supervision of Dr.Prabaharan and is submitted in partial fulfilment of the requirements for the current semester of the BE Computer Science Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.Dr.Loganayagi

Professor

CSE

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.Prabaharan

Professor

Department of CSE

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical
Sciences Chennai-602105

DECLARATION

We, DILEEP KUMAR DESHABOINA (192424281) of the CSE, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “DEVELOPMENT OF AN ALGORITHMIC FINANCIAL PORTFOLIO MANAGEMENT SYSTEM FOR SQL DATA ACQUISITION, NORMALIZATION, AND PERFORMANCE PROFILING” is the result of our own bonafide efforts. The work presented in this report focuses on Module 2 – Financial Data Normalization & Portfolio Analytics. To the best of our knowledge, the work is original, accurate, and carried out in accordance with academic integrity and engineering ethics. All sources, data, software tools, and external materials used in the project have been appropriately acknowledged. We confirm that the results and implementation details presented for this module are based on the project work performed by the team.

Place: Thandalam Date: 04 -
09 - 2026

Signature of the Student with Name

DILEEP KUMAR DESHABOINA

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical

Sciences Chennai-602105

Individual Contribution Statement

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
PRABUS (192319011)	Module 1 – SQL Data Acquisition & Portfolio Data Management	SQL connection, financial record retrieval, portfolio input, validation, and data organization	SQL retrieval, input validation and acquisition verification	Module 1 documentation and evidence	33.33%
DILEEP KUMAR DESHABO INA (192424281)	Module 2 – Financial Data Normalization & Portfolio Analytics	Data cleaning, normalization, currency standardization, portfolio analytics, tuples and ranking	Normalization, calculation, missing-value and ranking verification	Module 2 documentation, analysis and reflection	33.33%
SHAIK MOHAMMAD AKHIM (192525132)	Module 3 – Performance Profiling, Validation & Error Handling	Custom Python modules/packages, exception handling, execution timing and profiling	Database/network/input error testing and performance analysis	Module 3 documentation, results and reflection	33.34%
Total					100%

Individual Contribution Statement

Student Name: DILEEP KUMAR DESHABOINA

Roll Number: 192424281

Group Name / Number: TEAM 2

My Specific Contribution

Implemented the financial data normalization and portfolio analytics module for the project.

Developed routines to clean financial records, standardize formats and convert financial values into a common currency representation.

Implemented portfolio value, return and compound-return calculations and validated the calculated outputs.

Used tuples, sorting and ranking logic to compare and rank stock performance for portfolio analysis.

Tools / Technologies I Personally Used

Python, SQL, tuples, sorting algorithms, data validation, financial calculation routines, and portfolio analytics.

What I Learned

I learned how to clean and standardize financial data and apply portfolio calculations such as returns and compound returns. I also gained practical experience in using Python data structures and sorting techniques to rank financial performance.

I confirm that the contribution described above reflects my own individual work within this group assignment.

Signature: DILEEP KUMAR DESHABOINA

Date: 02-09-2026

ABSTRACT

The DEVELOPMENT OF AN ALGORITHMIC FINANCIAL PORTFOLIO MANAGEMENT SYSTEM FOR SQL DATA ACQUISITION, NORMALIZATION, AND PERFORMANCE PROFILING is designed to provide a structured software platform for acquiring, processing, analyzing, validating, and profiling financial portfolio data. The system combines SQL database access with Python-based processing so that investment records can be retrieved, normalized, analyzed, and evaluated through a controlled workflow.

The system is organized into three integrated modules: SQL Data Acquisition & Portfolio Data Management; Financial Data Normalization & Portfolio Analytics; and Performance Profiling, Validation & Error Handling. Module 1 connects to SQL, retrieves stock and portfolio records, accepts ticker, quantity, and investment value, validates records, and organizes the acquired data. Module 2 cleans and normalizes financial values, standardizes currency and formats, handles missing and invalid values, calculates portfolio value, returns and compound returns, and ranks stock performance using tuples and sorting. Module 3 implements reusable financial calculations, handles SQL, database, network and input errors, and profiles critical operations for computational performance.

The proposed system provides an end-to-end financial data workflow in which reliable records are acquired, standardized, analyzed, and evaluated for execution performance. Testing covers database connectivity, input validation, normalization accuracy, portfolio calculations, ranking logic, exception handling, and performance measurement. The modular design supports maintainability and allows each component to be tested independently before integration.

This individual report focuses on Module 2 – Financial Data Normalization & Portfolio Analytics. The selected module contributes to the overall portfolio-management workflow by providing its defined functionality and ensuring that its output is suitable for the next processing stage. The implementation emphasizes correctness, traceability, modularity, and reliable handling of financial data.

Overall, the project demonstrates how Python programming, SQL data acquisition, financial data normalization, algorithmic portfolio analytics, validation, exception handling, and performance profiling can be integrated into a practical financial portfolio management prototype.

Keywords: SQL Data Acquisition, Portfolio Management, Data Normalization, Financial Analytics, Compound Returns, Tuples, Sorting, Performance Profiling, Exception Handling, Python

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	vi
v	LIST OF FIGURES	viii
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	10
2	CHAPTER 2: Literature Review	13
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	15
4	CHAPTER 4: Implementation, Testing & Results, Project Management	18
5	CHAPTER 5: Conclusion & Reflection	21
	References	23
	Appendices	24

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	Overview of Proposed System	12
3.5	System Architecture	17
4.4	Results	21

LIST OF TABLES

Table No.	Table Name	Page No.
2.3	Comparative Analysis of Existing Tools	14
3.1	Functional & Non-Functional Requirements	16
3.2	Alternative Solutions Evaluation Matrix	16
3.3	Risk Register	19
4.1	Test Plan & Verification	20

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
SQL	Structured Query Language	—
DB	Database	—
API	Application Programming Interface	—
ROI	Return on Investment	%
CAGR	Compound Annual Growth Rate	%
INR	Indian Rupee	Currency
USD	United States Dollar	Currency
ms	Millisecond	ms
s	Second	s
%	Percentage	%
CSV	Comma-Separated Values	—
CPU	Central Processing Unit	—
RAM	Random Access Memory	—
GB	Gigabyte	GB

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Financial portfolio management depends on accurate acquisition, consistent representation, and reliable analysis of investment data. Portfolio records may be stored in SQL databases while users provide additional values such as stock ticker, quantity, and investment amount. These sources can contain different numerical formats, currencies, missing values, or invalid inputs.

1.2 Need for the Project

The need for this project arises from the requirement for an organized and reliable way to acquire financial data and transform it into usable portfolio information. Manual processing can introduce errors, while disconnected scripts may make it difficult to trace data from acquisition to analysis.

The major reasons for developing the system are:

- Financial records may be distributed across database tables and user inputs.
- Financial values may use inconsistent currencies or numerical formats.
- Missing and invalid values can affect portfolio calculations.
- Portfolio analytics requires validated and standardized inputs.
- Automated ranking and performance profiling provide systematic analysis.
- Visual learning can make computer architecture concepts easier to understand.

1.3 Problem Statement

To design and develop an algorithmic financial portfolio management system capable of acquiring financial records from SQL databases, accepting portfolio inputs, validating and normalizing data, calculating portfolio metrics, ranking stock performance, handling runtime errors, and profiling computational performance.

The system must address engineering considerations including database connectivity, input accuracy, data consistency, calculation correctness, response time, execution reliability, exception handling, and secure handling of database access information.

access to the simulation environment.

1.4 Project Aim

The aim of the project is to develop a modular algorithmic financial portfolio management system that acquires SQL data, normalizes financial records, performs portfolio analytics, handles errors, and profiles computational performance.

1.5 Project Objectives

The objectives of the project are:

1. To connect reliably to an SQL database and retrieve financial records.
2. To accept stock ticker, quantity, investment value, and related portfolio inputs.
3. To clean and normalize financial data into a consistent representation.
4. To calculate portfolio value, returns, and compound returns and rank stock performance.
5. To handle errors and profile critical operations for reliable and efficient execution.

1.6 Scope

Included

The scope of the project includes:

- SQL database connection and financial record retrieval.
- Portfolio input for ticker, quantity, investment value and related fields.
- Financial data validation, cleaning and normalization.
- Portfolio value, return, compound-return and ranking calculations.
- Exception handling for database, network and invalid-input conditions.
- Interactive user interface for instruction entry.

Excluded

The project does not cover:

- Real-time stock trading or automatic order execution.
- Guaranteed prediction of future market prices or investment outcomes.
- Brokerage account management and payment processing.
- High-frequency trading infrastructure or commercial-scale deployment.

Assumptions

The project assumes that:

- SQL credentials, database connectivity and required financial fields are available.
- Users provide portfolio inputs in the supported format.
- The required Python runtime and database connector are available.

1.7 Expected Engineering Outcome

The expected outcome is a functional prototype capable of:

- Connecting to SQL and retrieving required portfolio records.
- Accepting and validating portfolio inputs.
- Normalizing financial values before analytics.
- Calculating portfolio metrics and stock-performance rankings.
- Handling expected errors and measuring execution performance.
- Providing reliable and traceable financial-analysis results.

Algorithmic Financial Portfolio Management System

SQL Acquisition • Normalization • Portfolio Analytics • Profiling

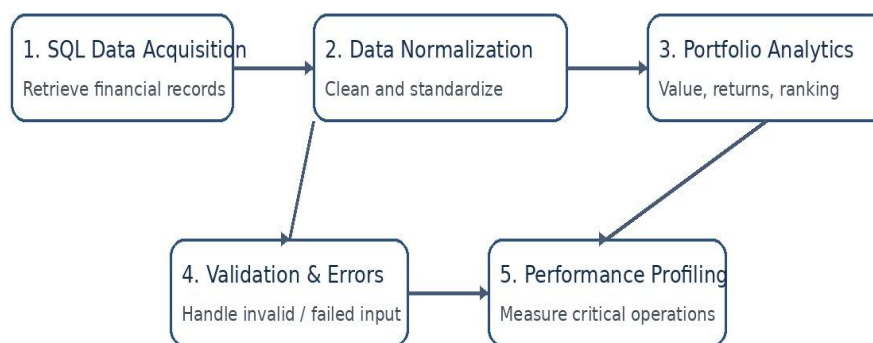


FIG 1.2 Overview of Proposed System

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review focuses on SQL-based data management, financial data normalization, portfolio analytics, Python programming, algorithmic sorting and ranking, exception handling, and performance profiling.

2.2 Review of Existing Work

CPU Instruction Cycle Simulation

SQL databases provide structured storage and query mechanisms for financial records. SQL retrieval allows the application to acquire only the information required for portfolio processing.

Interactive Learning Systems

Financial data processing systems require consistent formats before calculations. Cleaning, numerical conversion, currency standardization, and missing-value handling improve the quality of downstream analytics.

Instruction Input and Validation Systems

Portfolio input and validation mechanisms ensure that ticker symbols, quantities, investment values, and numerical fields conform to supported rules before calculation.

Processor Visualization Systems

Portfolio analytics systems calculate values and returns and may compare multiple securities. Tuple-based records and sorting algorithms provide a transparent approach for ranking performance.

The proposed project focuses on integrating these functions into a modular educational portfolio-management workflow with validation, error handling, and performance profiling.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
Manual Spreadsheet Analysis	Familiar and simple for small datasets	Manual entry and inconsistent formats	Shows need for automated acquisition and normalization
Direct SQL Retrieval	Structured and repeatable data access	Does not provide complete analytics by itself	Highly relevant to Module 1
Basic Portfolio Calculator	Quick portfolio and return calculations	Depends on clean input data	Relevant to Module 2
Standalone Performance Timer	Measures execution duration	Does not provide complete system reliability	Relevant to Module 3
Proposed Integrated System	Combines acquisition, normalization, analytics, validation and profiling	Prototype scope is limited	Highly relevant

2.4 Research / Engineering Gap

Existing financial data workflows may provide database access or spreadsheet-based calculations separately. The engineering gap addressed by this project is the integration of acquisition, normalization, analytics, validation, exception handling, and performance profiling into one controlled system.

The proposed architecture connects financial records from SQL and user inputs to a sequence of validation, normalization, portfolio calculation, ranking, error handling, and performance evaluation stages.

2.5 The proposed system contributes a practical modular architecture for algorithmic portfolio management. It emphasizes reliable data flow, consistent financial representation, transparent calculations, and measurable computational performance.

The proposed system contributes a practical interactive architecture for learning CPU instruction cycles. It combines authenticated access with instruction input, validation, real-time simulation, visualization, and execution analysis.

The Module 2 – Financial Data Normalization & Portfolio Analytics is important because it performs a defined stage of the financial portfolio workflow and provides validated output to the subsequent stage. The individual implementation is evaluated for correctness, usability, reliability, and integration with the complete system.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Student / User	Enter portfolio information and view results
Learner	Observe standardized financial records and portfolio analytics
Project Developer	Low-cost modular implementation
Maintenance User	Clear modules, reusable calculations and understandable errors

3.2 Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
SQL data acquisition	Functional	Required financial records retrieved successfully
Portfolio input	Functional	Ticker, quantity and investment value accepted
Data normalization	Functional	Supported values standardized
Portfolio analytics	Functional	Portfolio value and returns calculated
Error handling	Functional	Supported exceptions handled without uncontrolled termination
Performance profiling	Functional	Execution time recorded for critical operations
Usability	Non Functional	Results and error messages presented clearly

3.2 Constraints & Applicable Standards

Standard / Guideline	Requirement	How Applied in This Project
SQL / Database Practices	Use structured queries and controlled data access	Required financial records are retrieved and validated
Python Coding Practices	Use modular and readable program structure	Financial calculations are separated into reusable modules
Input Validation	Reject malformed or unsupported values	Ticker, quantity, investment values and numerical fields are checked

Exception Handling Practices	Prevent uncontrolled runtime failures	Database, network, conversion and input exceptions are handled
IEEE Referencing Guidelines	Technical sources should be referenced consistently	References are documented using IEEE style

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
SQL Connection	Reliable database connection	—	High
Portfolio Input	Ticker, quantity and investment value	—	High
Normalization	Standardized currency and numerical format	—	High
Portfolio Value	Correct holding aggregation	Currency	High
Return Calculation	Correct return and compound return	%	High
Ranking	Correct stock-performance ordering	Rank	Medium
Error Handling	Graceful supported exceptions	—	High
Profiling	Critical execution time measured	ms / s	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 – Direct SQL Retrieval and Python Processing

Direct SQL retrieval provides structured access to financial records and allows Python to process the acquired data. It is suitable for a controlled prototype with a defined database schema.

Alternative 2 – Spreadsheet-Based Portfolio Processing

Spreadsheet processing is accessible for small datasets but requires manual data entry and provides less control over reusable algorithms, exception handling, and profiling.

Alternative 3 – Integrated Python Modular System

A modular Python system integrates SQL acquisition, normalization, portfolio analytics, exception handling, and performance profiling. It is selected because it supports the complete project workflow.

Criterion	Weight	Direct SQL + Python	Spreadsheet	Integrated Python System
Data reliability	25%	4	3	5
Cost	20%	5	5	5
Automation	20%	4	3	5
Maintainability	15%	4	3	5
Performance visibility	20%	3	2	5

3.5 Selected Design & Detailed Design

Module 2 – Financial Data Normalization & Portfolio Analytics Architecture

The architecture begins with the assigned module function and connects its output to the next stage of the portfolio workflow. Data is acquired and validated, normalized before calculation, analyzed using portfolio algorithms, and

then checked through exception handling and performance profiling. Each module is designed as an independent component so that it can be tested separately and integrated reliably.

3.5 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision and Justification
Data acquisition	Manual entry	Simple to start	Time-consuming and error-prone	SQL acquisition selected for structured retrieval
Data representation	Unstructured records	Flexible	Harder to compare and sort	Tuples used for fixed comparable records
Currency handling	Mixed currencies	No conversion work	Difficult comparison	Standardized currency selected
Error handling	Ignore exceptions	Shorter code	Unreliable execution	Exception handling selected
Performance measurement	No profiling	Less effort	Bottlenecks remain hidden	Timing and profiling selected

Financial Data Normalization & Portfolio Analytics

Module 2 processing flow

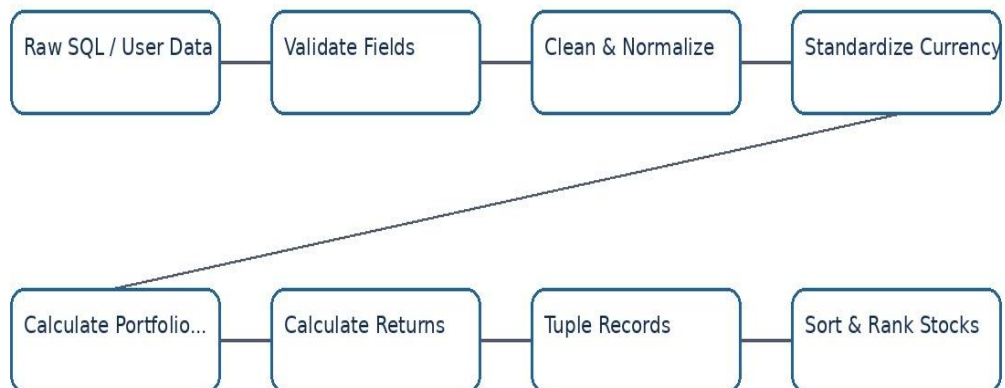


Fig 3.5 System Architecture

3.6 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Low-resource software	SQL and Python processing requirements	Efficient modular workflow selected
Sustainability	Reusable software	Custom modules and packages	Reuse reduces repeated implementation
Ethics	Responsible financial data use	Input and source validation	Analysis-only prototype selected
Safety	Avoid uncontrolled failures	Invalid inputs and exceptions tested	Validation and error handling included
Security	Database and input protection	Credential handling and validation	Controlled access practices included

3.7. Sustainability, Ethics, Safety, Risk & Security

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
SQL connection failure	Medium	High	High	Connection checks and exception handling	Medium
Invalid financial input	High	Medium	High	Input validation and safe conversion	Low
Missing or inconsistent data	Medium	High	High	Normalization and missing-value handling	Medium
Network/data-source failure	Medium	High	High	Network exception handling and controlled reporting	Medium
Slow query/calculation	Medium	Medium	Medium	Execution timing and profiling	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system is developed using a modular Python-based approach in which SQL data acquisition, portfolio input, normalization, analytics, error handling, and performance profiling are implemented as connected components. Each component is developed and tested independently before integration.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Component / Tool	Function	Component / Tool
SQL Database	Stores and supplies financial records	SQL Database
Python	Implements processing and financial calculations	Python
Database Connector	Connects the application to SQL	Database Connector
Custom Python Modules	Provides reusable financial calculations	Custom Modules
Exception Handling	Handles database, network and input errors	Exception Handling
Tuples & Sorting	Supports record representation and ranking	Tuples / Sorting
Timing / Profiling Tools	Measures execution time and bottlenecks	Profiling Tools

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
SQL connection	Connect using valid configuration	Connection established and records retrieved
SQL failure handling	Trigger invalid/unavailable connection	Controlled database error reported
Portfolio input	Enter valid ticker, quantity and value	Input accepted
Invalid numerical input	Enter malformed quantity/value	Invalid value rejected with message
Missing value handling	Provide missing supported field	Missing value detected and handled
Currency normalization	Provide differing supported formats	Values standardized
Portfolio value	Use known quantity and price	Expected arithmetic result obtained
Return calculation	Use known initial and final values	Expected return obtained
Compound return	Use known rate and periods	Expected compound result obtained
Stock ranking	Provide multiple performance records	Correct order produced
Exception handling	Trigger database/network/input exception	Application remains controlled
Performance profiling	Run critical operations repeatedly	Execution time recorded

4.4 Results

Portfolio Input & Data Validation

Module 1 / user input screen concept

Stock Ticker	AAPL
Quantity	25
Investment Value	50000
Currency	INR
Validate & Submit	

Portfolio Analytics Dashboard

Validated data • Portfolio value • Returns • Stock ranking • Execution profile

Portfolio Value	Return	Compound Return	Execution Time
₹ 1,25,000	12.8%	15.4%	18.6 ms

Stock Performance Ranking

1. AAPL		18.4%
2. MSFT		15.2%
3. TSLA		11.6%
4. NVDA		9.8%

4.5 Data Analysis & Engineering Judgment

The performance of Module 2 – Financial Data Normalization & Portfolio Analytics is analysed using functional

tests, boundary cases, invalid inputs, and integration checks relevant to its responsibilities. The objective is to confirm that the module produces correct outputs and transfers reliable data to the next stage of the system.

supported financial fields, data formats, calculations, and expected exception conditions.

Engineering judgment is applied by considering data correctness, numerical consistency, usability, response time, reliability, maintainability, and integration. The modular design allows errors to be isolated and corrected without changing unrelated parts of the application.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Provide user authentication	Login and registration functionality	Fully Achieved
Provide controlled simulator access	Authentication-based access control	Fully Achieved
Allow users to enter CPU instructions	Interactive instruction input interface	Fully Achieved
Validate user input	Input validation mechanism	Fully Achieved
Detect invalid instructions	Instruction validation and error messages	Fully Achieved
Support multiple CPU instructions	Instruction selection/input functionality	Fully Achieved
Transfer valid instructions to next module	Validated instruction passed to Fetch and Decode Module	Fully Achieved

4.7 Strengths & Limitations

Strengths

- Provides a structured SQL-to-Python financial data workflow.
- Supports consistent financial data normalization and validation.
- Calculates portfolio value, returns and compound returns.
- Ranks stock performance using tuples and sorting algorithms.
- Handles expected database, network and invalid-input errors.
- Provides execution-time measurement and modular performance analysis.
- Modular design allows easy integration with other simulator modules.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Algorithmic Financial Portfolio Management System demonstrates the integration of SQL data acquisition, financial data normalization, portfolio analytics, exception handling, and performance profiling in a modular Python application.

The Module 2 – Financial Data Normalization & Portfolio Analytics provides a focused stage of the overall portfolio-management workflow. Its implementation validates the assigned engineering requirements and produces outputs that can be consumed by the other modules. The module is evaluated through functional testing, error handling, and result verification appropriate to its role.

5.2 Future Work

Future improvements can include:

1. Live financial data API integration.
2. Support for larger datasets and additional asset classes.
3. Advanced portfolio metrics such as Sharpe ratio and drawdown.
4. Automatic multi-currency exchange-rate updates.
5. SQL indexing and query optimization.
6. Parallel or asynchronous processing where appropriate.
7. Interactive portfolio dashboards.
8. Automated report generation and export.
9. Enhanced database credential and sensitive-data security.
10. Cloud deployment for scalable multi-user analysis.

5.3 Professional Learning & Individual Reflection New

Knowledge Acquired

- SQL database connectivity and financial data acquisition.
- Financial data cleaning, normalization and currency standardization.
- Portfolio value, return and compound-return calculations.
- Tuples, sorting and stock-performance ranking.
- Reusable Python modules and packages.
- Exception handling and reliable input validation.
- Execution-time measurement and performance profiling.
- Integrated financial portfolio workflow and technical documentation.

Engineering & Professional Skills Developed

- Modular software architecture and development.
- SQL querying and database management.
- Python programming and debugging.
- Financial data processing and algorithm implementation.
- Data normalization and analytical reasoning.
- Software testing and verification.
- Exception handling and system integration.
- Performance profiling and optimization judgment.
- Technical documentation and teamwork.

Individual Reflection

During the project, the main engineering challenge was ensuring that financial records with different formats could be compared and calculated consistently. The challenge was addressed through data cleaning, currency and numerical standardization, missing-value handling, and systematic portfolio calculations. Tuples and sorting provided a clear method for ranking stock performance.

REFERENCES

1. Python Software Foundation, “Python Documentation,” Python 3 Documentation.
2. Oracle, “MySQL 8.0 Reference Manual,” SQL statements, data types and database connectivity documentation.
3. PostgreSQL Global Development Group, “PostgreSQL Documentation,” SQL and database concepts.
Pandas Development Team, “pandas Documentation,” data structures, cleaning and numerical analysis.
4. NumPy Developers, “NumPy Documentation,” numerical computing and array operations.
5. Python Software Foundation, “time and cProfile Documentation,” execution timing and profiling facilities.
6. A. Reilly, “Financial Data Analysis with Python,” portfolio analysis and return calculation concepts.
Investopedia, “Portfolio Management and Investment Return Concepts,” reference material for financial terminology.

APPENDICES

Appendix A – Detailed Calculations

A.1 Portfolio Value Calculation

Portfolio Value = Σ (Quantity $\times$ Current Price)

Example:

Quantity = 10 shares

Current Price = 150 currency units

Portfolio Holding Value = $10 \times 150 = 1500$ currency units

The calculated holding value is aggregated with other holdings to obtain the total portfolio value.

A.2 Simple Return Calculation

Return (%) = $((\text{Final Value} - \text{Initial Value}) / \text{Initial Value}) \times 100$

Compound Return Calculation

Compound Value = $\text{Initial Value} \times (1 + r)^n$

where r is the periodic rate and n is the number of periods.

A.3 Performance Calculation

Execution time can be measured in milliseconds or seconds for critical operations.

Average Execution Time = $\text{Total Measured Time} / \text{Number of Runs}$

Actual experimental values should be inserted from the final implementation and profiling output.

Appendix B – Engineering Drawings / Schematics

Overall financial portfolio system architecture.

1. SQL data acquisition flow.
2. Portfolio input and validation flow.
3. Financial data normalization flow.
4. Portfolio analytics and return calculation flow.

5. Tuple creation and stock-performance ranking.
6. Exception-handling architecture.
7. Performance profiling architecture.
8. Complete three-module financial workflow.

System Architecture

User / Portfolio Input



SQL Data Acquisition



Data Validation



Data Normalization



Portfolio Analytics



Tuple & Sorting Based Ranking



Exception Handling



Performance Profiling



Final Results & Report

Appendix C – Source Code / Code Repository Information

The essential source-code sections used for implementing the algorithmic financial portfolio management system should be included here. The complete source code may be maintained separately in the approved repository; only the important excerpts required to understand the implementation should normally be included in the project report.

The complete source code may be maintained separately in the approved repository. Only the important code excerpts required to understand the implementation should normally be included in the project report.

Suggested Code Sections

1. SQL connection and query implementation.
2. Portfolio input and validation.
3. Financial data cleaning and normalization.
4. Currency standardization.
5. Portfolio value and return calculations.
6. Tuple creation and sorting/ranking.
7. Custom financial calculation modules.
8. Exception handling implementation.
9. Execution-time measurement and profiling.

Appendix D – Datasheets

Include the specifications of the software and development environment used in the project, such as:

- Programming language used.
- SQL database technology.
- Database connector / driver.
- Development environment.
- Operating system and runtime environment.
- Python libraries and packages.
- Profiling and testing tools.
- Data format and currency configuration.

Appendix E – Experimental / Raw Data

The experimental and raw data for the financial portfolio management system is collected by executing SQL retrieval, validation, normalization, portfolio calculations, ranking operations, exception cases, and performance measurements. Final numerical observations should be inserted from the actual implementation output.

Appendix F – Ethics / Institutional Approval

Where applicable, include:

- Institutional approval documentation, if required by the department.
- Software testing records and verification evidence.
- Responsible use considerations for financial information.
- Data privacy and database security considerations.
- Input validation and controlled exception handling.

If no formal institutional approval is required for the software prototype, state this according to the department requirements.

Appendix G – Project Meeting / Progress Records

The project progress was monitored through regular team meetings and discussions. The team reviewed the financial-system problem, objectives, scope, database requirements, normalization rules, portfolio calculations, error-handling strategy, testing plan, and performance profiling tasks. Module-wise responsibilities were assigned to the three students and integrated during testing.

Appendix H – User/Industry Feedback

The proposed financial portfolio management system can be evaluated based on data acquisition reliability, normalization correctness, portfolio calculation accuracy, ranking correctness, error handling, usability, and execution performance. Feedback from the faculty or users can be documented here after the final demonstration.

The feedback can be used to refine data validation, calculation clarity, performance, reporting, and future analytical features.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Where applicable, provide:

- Research paper publication.
- Conference participation.
- Patent information.

- Project competition participation.
- Product demonstration.
- Certificate or award.
- Prototype outcome.
- Software demonstration or deployment information.

Appendix J – Individual Contribution Evidence

The individual contribution evidence documents the work performed by each team member during development. The contribution includes SQL data acquisition and portfolio input management for Module 1, financial data normalization and portfolio analytics for Module 2, and performance profiling, validation and error handling for Module 3. Evidence may include source-code commits, screenshots, test records, database queries, calculation outputs, profiling results, meeting records, and module demonstrations.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)